

現場 Top 6 速答

自行訓練：我們訓練的是 task-specific confidence calibrator，不是另訓大型語言模型。

為何不用現成模型：Bedrock 負責語言理解；信任分數與校準需要可重現、可稽核的 deterministic pipeline。

ModelHub / SageMaker：[ModelHub](#) 管治理，[SageMaker](#) 執行 AWS 訓練。

AWS 限制：生成式 foundation model 只走 Bedrock；校準器是任務模型，正式路徑可走 [SageMaker](#)。

來源驗證：台灣監管 connector 已建立，但各來源 coverage 依 live、cache、fixture 狀態揭露；BlockTempo 仍是 planned。

15 分鐘失敗：以 budget、cache、平行來源與 fail-closed 控制；資料不足就 abstain，不硬湊答案。

TrustForge Hermes | 商業化產品備詢

20 題深度問答 | 產品、工程、金融治理與商業化邊界

總原則：回答先講產品決策，再講實作證據，最後講限制與下一步。不要把「已沉澱 learning event」說成「每次 run 立即自動訓練」；不要把「candidate」說成「已上線模型」；不要把「可替換介面」說成「所有後端都已在 production 啟用」。

產品與商業化定位

1. TrustForge 和一般 LLM、RAG 或 OpenClaw Agent 的根本差異是什麼？

一般 LLM 解決生成，一般 RAG 解決找資料，Agent framework 解決工具調用與行動編排。TrustForge 解決的是高風險資訊工作的 trust boundary：每個主張有 evidence contract，每個 run 有 execution lineage，每個分數有 deterministic reasoning，每次模型或 policy 變更有 candidate、review、activation 與 rollback。它不是另一個聊天模型，而是把 Agent 的執行結果變成可被金融企業採用的 decision record。

```
`src/trustforge/pipeline.py`、`schema.py`、`execlog.py`、`trust/kernel.py`、`upgrade_control.py`、  
`frontend/src/components/AnalysisReportView.tsx`。
```

2. 商業客戶實際買到什麼？

客戶買到三個面向：第一，研究 workspace，把多源市場訊號轉成可查證報告；第二，governance console，觀測成本、資料品質、模型候選、升級與 incident；第三，API 與 artifacts，讓 report、evidence、execution log、snapshot 和 telemetry 接入風控、客服、研究或稽核系統。價值不是多一個 dashboard，而是縮短從資訊取得到可批准決策的時間，並降低黑箱與事後追責成本。

TrustForge 不下單、不提供買賣建議，也不承諾報酬。

Runtime 與資料管線

3. 一次 analysis run 的完整生命週期是什麼？

建立 run_scope_id 後，Ingestion 依來源政策取得資料；claim extraction 把文件轉成可追溯主張；dedup 與 corroboration 判斷來源獨立性；Trust Kernel 計算來源信譽、時效、stance、操縱風險與信心；pipeline 產出 report、evidence 與 execution log；前端再以 Evidence Trail、facts/inferences ladder、contrarian、could_flip 和 technical execution journey 呈現。

```
`pipeline.run()`、`agent/orchestrator.py`、`trust/scoring.py`、`trust/kernel.py`、  
`frontend/src/components/AnalysisReportView.tsx`。
```

4. 你們有哪些資料來源與 connector ?

資料面包含 OHLCV 與 HOYA BIT、價格交叉來源、新聞/RSS、社群、官方與監管、鏈上交易、Whale Alert、Arkham、Etherscan、CoinMarketCap、CoinGecko、DefiLlama，以及台灣監管來源。每類 connector 都有 safe fetch、cache、reliability、fetched_at 與 degraded handling；來源不是越多越好，而是要能辨識、去重、回源與說明資料模式。

```
`src/trustforge/ingestion/` 下的 `prices.py`、`hoyabit.py`、`news.py`、`social.py`、`regulatory.py`、  
`onchain.py`、`whale_trades.py`、`cmc.py`、`coingecko.py`、`defillama.py`。
```

5. 如何避免同一篇新聞被重複計票？

系統先建立 canonical source identity，再對 claim 做分組與相似度比對；同稿轉載會落到同一個 evidence group，不會因為 URL 不同就被當成多個獨立來源。[Trust Kernel](#) 計算 independent_sources，而不是 raw document count；來源衝突則保留 stance pair 和 contrarian evidence，不靜默刪除。

```
`agent/evidence_grouper.py`、`trust/scoring.py`、`src/trustforge_core/corroboation.py`、  
`frontend/src/lib/evidenceGrouping.ts`。
```

6. Bedrock 在你們的產品中到底負責什麼？

Bedrock 是唯一生成式模型入口，主要負責非結構化文件的 claim extraction、有限的 semantic stance classification，以及把 pipeline 已經決定的 facts、inferences、evidence 和 limits 組成可讀 narrative。它不直接決定 source weight、不直接產生市場方向、不取代 [Trust Kernel](#)，也不能把自己的新依據偷偷加入報告。

若 Bedrock 不可用，系統可在 offline/sample 或 deterministic fallback 模式輸出明確降級結果；不把 fallback 偽裝成 live。

Trust Domain 與風險語意

7. TrustScore 是預測準確率嗎？

不是。TrustScore 衡量的是本輪 evidence 的來源可靠度、交叉佐證、時效與操縱風險；calibrated confidence 是校準後的相對信心，不是統計 coverage 保證，也不是投資報酬或漲跌準確率承諾。當 evidence 不足或獨立來源太少，decision state 會進入 low-confidence 或 abstain。

8. Trust Kernel 現在是否真的完全零 I/O、獨立可拔插？

Trust Kernel 已有 frozen KernelInput/KernelOutput contract 和 `run_kernel()`，設計上把 IO、AWS、HTTP、cache 與 provider 排除在核心計算之外；但架構文件也誠實標註，現行 production pipeline 仍有部分路徑直接使用 `scoring.py`、BedrockClient 與 ingestion。商業化 roadmap 是讓所有 production composition 逐步收斂到 kernel entry point，而不是在簡報中宣稱尚未完成的 SLA。

``docs/architecture/ARCHITECTURE-OVERVIEW.md``、``docs/architecture/TRUST-KERNEL-BOUNDARY.md``。

9. Asset Intrinsic、Peer Metrics、Whale 與 Carbon 不是額外噓頭嗎？

它們是不同風險問題的 context plane。Asset Intrinsic 提供資產基本面與治理脈絡，Peer Metrics 支援跨幣比較，Whale Activity 捕捉大額鏈上異動，Carbon 记录 token、成本與碳排估算。它們不直接覆寫 TrustScore，而是作為 evidence、risk flag、comparison 與 operational decision 的補充視角。

``asset_intrinsic.py``、``peer_metrics_scheduler.py``、``whale_api.py``、``carbon.py``、``frontend/src/components/AssetIntrinsicShadowPanel.tsx``。

Learning、ModelHub 與 SageMaker

10. 你們自行訓練的是什麼模型？

我們自行訓練的是任務型 calibration model，主要是 Isotonic Regression，也保留 logistic calibrator candidate。輸入是 raw confidence、coin、direction 等 feature，標籤是延遲成熟的 market outcome；輸出是 calibrated confidence 與可比較的 calibration metrics。這不是重新訓練 Claude，也不是自己訓練一個通用聊天模型。

11. 為什麼不只使用 AWS 現成模型？

Bedrock 的 foundation model 擅長語意理解與生成，但不知道我們的 evidence contract、source reliability、freshness half-life、dedup semantics 或金融市場的 delayed ground truth。自有 calibrator 的目的不是取代 Bedrock，而是讓模型輸出的信心符合 TrustForge 的任務與資料分布；這能降低 over-confidence，並讓同一套治理標準跨模型、跨 provider 維持一致。

12. 每次分析完成後是不是立即自動訓練？

產品設計是 closed-loop learning，但不是立即重訓。每次 run 會寫入 learning event、raw confidence、evidence 與 execution metadata；要等 $T+1/T+7/T+14$ outcome label 成熟，才能形成沒有 label leakage 的 training row。接著由 `prepare\_calibrator\_training.py`、training contract、calibrator gate 和 backend submit 流程產生 candidate。現況訓練觸發是顯式 CLI；自動排程是規劃中的 enhancement，activation 永遠由人核准。

| `docs/reports/REPORT-2026-07-31-TRAINING-AUTO-TRIGGER-ANALYSIS.md` 明確記錄目前沒有 scheduler。

13. ModelHub 和 SageMaker 的責任邊界？

ModelHub 是 governance surface：保存 model version、training request、metrics、artifact provenance、candidate status 與 approval lifecycle。SageMaker AI 是 execution backend：建立 Training Job、從 S3 讀資料、產出 artifact、回傳訓練結果。兩者透過 `TrainingBackend` contract 對齊，但不能把 ModelHub 說成 SageMaker，也不能把 candidate 說成 production。

14. 自行訓練是否違反只准 AWS foundation model ?

不違反，因為生成式 foundation model 仍只走 Amazon Bedrock。calibrator 是 TrustForge 的 deterministic task model，正式 AWS 路徑可以在 SageMaker AI 執行；它不呼叫第三方生成模型。若主辦方要求所有訓練也必須在 AWS，我們只啟用 SageMaker backend，[ModelHub](#) 留作治理與賽外介面。

Upgrade、AgentCore 與可靠性

15. Agent 會不會自己改 prompt、模型或 production code ?

不會。upgrade control 將 provider、policy、model、calibrator、RAG index 和 reranker 當成 candidate；流程是 diagnostics、sandbox、adversarial review、artifact integrity、reviewer gate、human activation、rollback。[Trust Kernel](#) 是 reviewed code release，不允許 outer skill 遞迴修改或自動 merge。

```
`upgrade_control.py`、`upgrade_queue.py`、`upgrade_review.py`、`policy/`、`docs/architecture/UPGRADE-CONTROL-OVERVIEW.puml`。
```

16. AgentCore、Memory、Policy 與 Hermes 的關係？

Hermes 是 TrustForge 的 domain agent product；AgentCore 是可替換的 runtime/provider integration。Memory plane 管 session、run、evidence 與 learning event 的讀寫邊界；Policy plane 編譯與執行來源、分析、報告、評估和改善政策；AgentCore 不取得 TrustScore 的決策權，也不繞過 TrustForge 的 evidence contract。

17. 如何處理重複提交、逾時或半完成分析？

formal run 使用 idempotency key、lease、budget reservation 與 durable state；multi-angle batch 使用 atomic batch store、reconcile 和 terminal receipt。每個請求有 timeout、remaining budget 與 retry policy；來源失敗可以降級，但 report 必須把 limits 和 data mode 寫出。無法完成時保留 failed [Execution Log](#)，不提交假成功。

安全、部署與驗證

18. 現在的真實 AWS 架構是什麼？

公開入口是 EC2 t3.micro 上的 nginx，對外有 80/443；React/Vite 靜態資產由 nginx 提供，Python API 只聽 127.0.0.1:8080。Bedrock 是唯一 LLM 入口，S3 保存 deployment/artifact，IAM Instance Role 提供最小權限，SSM Session Manager 取代 SSH key，CloudWatch 收集運行紀錄。App Runner 是未採用的舊規劃，不是目前部署事實。

``docs/architecture/AWS-ARCHITECTURE.md``、``deploy/nginx.conf``、``deploy/deploy_ec2.sh``。

19. API Key、AWS credential 和資料安全如何處理？

長效 credential 不進 Git；EC2 使用 IAM Instance Role，維運使用 SSM，live model access 受 token gate、rate limit、budget guard 與 request accounting 約束。nginx 對外，Python API 只在 loopback；admin endpoints 需要獨立 admin token 並 fail-closed。公開 artifact 只輸出必要 evidence，不暴露 secret、stack trace 或內部連線細節。

20. 你們如何證明一次執行是真實且可重現？

四份交付物共享 run_id 與 claim_id：Final Report 說明結論與限制，Claim、網址與時間戳。Evidence List 指向 source、時間和引用，Execution Log 記錄工具、模型、資料模式、成本與錯誤，Source/Config 提供原始碼、依賴與重現步驟。artifact manifest 與 SHA-256 綁定內容；UI 也能從報告跳到 Evidence Trail、Execution Journey 與下載檔案。

``schema.py``、``execlog.py``、``execution_event_log.py``、``frontend/src/components/ReportDownloads.tsx``、``EvidenceTrailPanel.tsx``。

最後的商業化回答

我們有什麼護城河？

不是單一 prompt，而是 evidence contract、Trust Kernel、calibration、governance、replay 與 operational controls 的整體系統。

我們最誠實的限制？

ModelHub/SageMaker training backend 已有介面與流程，但無人值守自動訓練與自動 activation 尚未宣稱完成；這是治理選擇，不是遺漏。

商業導入第一步？

先把研究或市場資訊頁接上 [Trust Layer API](#)，再以來源歷史、outcome labels 和 audit records 擴大到風控與客服。

一句話收尾？

TrustForge 讓企業不只得到 AI 的答案，也能知道答案憑什麼、誰批准、何時失效，以及如何回復。